

INFO CMN1 – Mise à niveau JAVA

FICHE DE RÉVISION COMPLÈTE - CHAPITRE 1 : Généralités Java

1. Concepts Fondamentaux et Architecture (Java vs C++)

L'une des questions théoriques les plus classiques en examen de Master consiste à comparer l'exécution d'un programme Java et d'un programme C++.

- **C++ (Langage Compilé)** : La compilation du code source produit des instructions natives directement exécutables par le processeur de la machine réelle. Le code obtenu est très performant, mais non portable (il doit être recompilé pour chaque système d'exploitation).
- **Java (Langage Interprété / Portable)** : Le compilateur Java ne génère pas de code machine natif, mais un pseudo-code intermédiaire universel appelé **bytecode**. Ce bytecode est ensuite lu et exécuté par un interpréteur : la **Machine Virtuelle Java (JVM)**.
- **La Machine Virtuelle (JVM)** : C'est une simulation logicielle d'un ordinateur réel. Elle comprend les instructions du bytecode et les traduit à la volée pour les exécuter dans l'environnement réel courant. C'est ce mécanisme qui rend Java **portable** ("*Write Once, Run Anywhere*").
- **Historique et Atouts** : Développé par **Sun Microsystems** en 1990. Sa syntaxe est très proche du C++. Historiquement dédié à Internet grâce aux **applets** (programmes Java encapsulés dans des pages Web pour l'interactivité client/serveur). Il dispose d'une grande richesse d'APIs (réseau, bases de données, interfaces graphiques).

2. Les Écosystèmes et Outils de Développement (JDK)

Le cours distingue trois approches pour développer en Java :

1. **Le JDK de Sun (Java Development Kit)** : La référence officielle et gratuite. Il contient tous les outils indispensables à la réalisation et à l'exécution (compilateur `javac`, interpréteur `java`), mais n'intègre pas de véritable environnement graphique.
2. **Les Freewares / Sharewares (IDE Gratuits)** : Offrent une interface graphique par-dessus le JDK et intègrent des outils d'aide (générateurs d'interfaces). Exemples : **Eclipse**, **NetBeans**.
3. **Les Environnements Professionnels** : Très puissants mais payants. Exemples : **JBuilder 2** (Inprise/Borland), **Visual J++** (Microsoft).

3. Syntaxe de Base et Conventions d'Écriture (Sun)

Sur une copie d'examen sur table, le professeur pénalise l'écriture de code qui ne respecte pas les conventions officielles.

Les 3 types de commentaires :

- `//` commentaire : Commentaire sur une seule ligne.

- `/* commentaire */` : Commentaire multiligne.
- `/** commentaire */` : Commentaire spécial destiné au système **Javadoc** pour générer automatiquement une documentation API à partir du code source.

Les Règles de Nommage (Java Code Conventions) :

- **Classes et Interfaces** : Commencent par une **Majuscule**. En cas de mot composé, chaque mot commence par une majuscule (*CamelCase*). Exemples : Etudiant, EtudiantBoursier.
- **Méthodes et Variables** : Commencent par une **minuscule**. Si le nom est composé, les mots suivants prennent une majuscule. Exemples : nom, numeroInscription, afficher(), toString().
- **Constantes** (déclarées avec `static final`) : Écrites **entièrement en MAJUSCULES** avec des tirets du bas. Exemple : Integer.MAX_VALUE.

4. Les Types de Données Primitifs et le Transtypage (Casting)

Java possède des types primitifs dont la taille en mémoire est fixe, peu importe la machine :

- **Entiers** : byte (8 bits), short (16 bits), int (32 bits), long (64 bits).
- **Flottants (Réels)** : float (32 bits), double (64 bits).
- **Caractère** : char (16 bits).

 **Le Casting (Transtypage)** : Permet de forcer la conversion d'un type vers un autre (comme en C). Attention, passer d'un réel à un entier provoque une **troncation** (perte de la partie décimale).

- *Exemple syntaxique :*
- `double d = 15.78;`
- `int i;`
- `i = (int) d; // i vaudra 10 (troncation, pas d'arrondi)`

5. Les Structures de Contrôle (Instructions)

La syntaxe des structures conditionnelles et des boucles est calquée sur celle du C/C++.

- **L'instruction conditionnelle if / else :**
- `if (condition) {`
- *// instructions si vrai*
- `} else {`
- *// instructions si faux*
- `}`
- **La boucle for :**
- `for (initialisation; condition; expression) {`

- *// instructions répétées*
- }
- *// Exemple du cours :*
- for (int i = 0; i < 5; i++) {
- System.out.print("Salut!");
- }
- **La boucle while (Tant que - Condition à l'entrée) :**
- while (condition) {
- *// instructions*
- }
- **La boucle do / while (Condition à la sortie - Exécutée au moins 1 fois) :**
- do {
- *// instructions*
- } while (condition);
- **L'aiguillage switch** : Permet de tester plusieurs valeurs. **Attention** : Ne jamais oublier l'instruction break; à la fin de chaque cas, sinon le système exécute les blocs suivants sans tester la condition. Le mot-clé default: capture tous les cas non spécifiés.
- switch (expression) {
- case valeur1:
- *// instructions;*
- break;
- case valeur2:
- *// instructions;*
- break;
- default:
- *// instructions par défaut;*
- }

QUIZ ÉCLAIR - CHAPITRE 1 : Test de syntaxe et concepts

Pour valider cette mise à niveau, répondez à ces 3 questions rapides :

Question 1 (Architecture)

Qu'est-ce que le **bytecode** en Java ?

- A) Le code source écrit par le programmeur dans le fichier .java.

- B) Le code machine natif binaire directement exécuté par le processeur Intel ou AMD.
- C) Un pseudo-code intermédiaire généré par le compilateur et destiné à être lu par la JVM.
- D) Un script de configuration pour les pages Web.

Question 2 (Conventions de nommage)

Selon les conventions de Sun, comment doit être nommée une variable stockant la moyenne générale d'un étudiant ?

- A) MoyenneGenerale
- B) moyenne_generale
- C) moyenneGenerale
- D) MOYENNE_GENERALE

Question 3 (Instructions)

Soit le code suivant :

```
int x = 2;
switch(x) {
    case 1: System.out.print("Un ");
    case 2: System.out.print("Deux ");
    case 3: System.out.print("Trois "); break;
    default: System.out.print("Autre");
}
```

Qu'est-ce qui s'affiche à l'écran lors de l'exécution ?

- A) Deux
- B) Deux Trois
- C) Deux Trois Autre
- D) Deux Autre

Donnez-moi vos réponses (ex: 1A, 2B, 3C).

FICHE DE RÉVISION UNIQUE ET FINALE - CHAPITRE 2 : POO, Classes, Objets, Héritage, Exceptions & Interfaces

1. Rappels Théoriques de la POO

- **Principe fondamental** : Les données étant plus stables que les traitements, il est préférable de structurer un système par rapport à ses données plutôt que par rapport à ses fonctions. **En POO, tout est dirigé par les données.**
- **Les 3 piliers de la POO** :

1. **L'Encapsulation** : Regroupement des données (attributs) et des traitements (méthodes) au sein d'une même structure (l'objet) en protégeant l'accès direct aux données. L'extérieur doit passer par l'interface visible (les services publics) pour garantir l'intégrité de l'objet.
2. **L'Héritage** : Transmission des propriétés d'une classe mère vers une sous-classe par dérivation (spécialisation), offrant la possibilité de les enrichir successivement.
3. **Le Polymorphisme** : Faculté d'une même méthode à pouvoir s'appliquer à divers objets instanciant chacun des classes différentes.

Messages, Signatures et Liaisons

- Message = Objet Récepteur + Signature de méthode
- Signature = Nom de la méthode + Paramètres (nombre, types et ordre).
- **Liaison statique** : Le choix du code à exécuter est réalisé à la **compilation** (performant).
- **Liaison dynamique** : Le choix du code est réalisé à l'**exécution** (cas de Java). En cas de polymorphisme, la JVM parcourt de façon ascendante le graphe d'héritage. La première méthode dont la signature correspond au message est exécutée.
- **Redéfinition (Overriding)** : Réécrire le code d'une méthode héritée **sans modifier sa signature ni son type de retour**.
- **Surcharge (Overloading)** : Définir deux méthodes de **même nom mais avec des signatures différentes** (soit au sein d'une même classe, soit par rapport à une super-classe).

2. L'Héritage en Java et la classe Object

- **Héritage simple unique** : En Java, une classe ne peut hériter que d'**une seule et unique classe mère** via le mot-clé `extends` (Ex: `public class EtudiantBoursier extends Etudiant`). L'héritage multiple est interdit pour les classes.
- **La racine `java.lang.Object`** : Toutes les classes en Java héritent implicitement et de façon universelle de la classe `Object`. Elle fournit des méthodes fondamentales qu'il est courant de redéfinir :
 - `getClass()` : Connaître la classe exacte d'un objet.
 - `equals(Object obj)` : Comparer la valeur des objets (et non leurs références).
 - `toString()` : Retourne une représentation textuelle de l'objet. (Exemple d'examen : redéfinir `toString()` dans `Fraction` pour renvoyer `this.num + "/" + this.den`).

Les chaînages de constructeurs (`super` et `this`)

Dans les constructeurs d'une sous-classe, vous devez savoir manipuler le ciblage mémoire :

- **`super(...)`** : Permet d'appeler explicitement un constructeur de la super-classe (classe mère). **Règle absolue sur table** : s'il est présent, `super()` doit obligatoirement être la toute première instruction du constructeur. (Ex: `super(nom);`).
- **`this(...)`** : Permet d'appeler un autre constructeur de la *même* classe pour éviter la duplication de code (Ex: `this(nom, 0);`).

- **Accès au masquage via super** : Si une sous-classe redéfinit une méthode ou possède un attribut de même nom que la classe mère (masquage), la pseudo-variable `super` permet de forcer l'accès au membre masqué de la classe mère.

3. Abstraction : Classes Abstraites vs Interfaces

L'une des questions de synthèse les plus importantes de l'épreuve écrite porte sur la distinction stricte entre ces deux concepts d'architecture.

Classes Abstraites (abstract)

- Définies par l'introduction du mot-clé **abstract** au niveau de la classe (Ex: `abstract class FormeGeometrique`).
- **Méthode abstraite** : Précédée du mot-clé **abstract**, elle définit uniquement l'interface (la signature) de la méthode et **ne possède pas de corps** (pas d'accolades, se termine par un point-virgule ;).
- **Règles d'or** :
 1. Une classe abstraite contient **au moins** une méthode abstraite.
 2. Elle **ne peut jamais être instanciée** directement (interdit d'écrire `new ClasseAbstraite()`).
 3. Elle doit obligatoirement être dérivée par une sous-classe concrète qui fournit une implémentation à *toutes* les méthodes abstraites sous peine de devoir être déclarée abstraite à son tour.

Les Interfaces (interface)

- Déclarées avec le mot-clé **interface** (Ex: `public interface FormeGeometrique`). C'est une classe abstraite poussée à l'extrême dont l'instanciation directe est impossible.
- **Contenu strict** : Une interface ne peut contenir **que** :
 1. Des attributs statiques constants implicites (**static final**).
 2. Des méthodes publiques non implémentées (sans corps).
- **Règles d'or et flexibilité** :
 1. Une interface peut hériter d'autres interfaces via l'héritage simple.
 2. Une classe peut implémenter **une ou plusieurs interfaces** simultanément via le mot-clé **implements** (Ex: `public class Rectangle implements FormeGeometrique`). **C'est le mécanisme qui remplace l'héritage multiple en Java.**
 3. Lorsqu'une classe implémente une interface, elle a l'obligation stricte d'implémenter le corps de *toutes* les méthodes définies dans l'interface.

Application pratique du Polymorphisme par Interface (Exemple type d'examen) :

On peut déclarer un tableau en utilisant le type de l'interface comme référence commune, puis y stocker n'importe quel objet qui implémente cette interface :

```
FormeGeometrique[] formes = new FormeGeometrique[3]; // Déclaration légale d'un tableau de références
```

```
formes[0] = new Cercle(10); // OK car Cercle implements FormeGeometrique
```

```
formes[1] = new Carre(4);    // OK car Carre implements FormeGeometrique
formes[2] = new Rectangle(5,8); // OK car Rectangle implements FormeGeometrique
```

```
for (int i=0; i < formes.length; i++) {
    // La JVM utilise la liaison dynamique à l'exécution pour appeler la bonne méthode surface()
    System.out.println(formes[i].surface());
}
```

4. Protection des données (Visibility) & Objets

Les 4 Niveaux de Visibilité (Encapsulation)

- **public** : Accessible partout. Hérité.
- **private** : Accessible uniquement au sein de sa propre classe. **Non hérité**.
- **protected** : Accessible par les classes du même package. Hérité par les sous-classes (même dans des packages distincts).
- **Par défaut** (aucun mot-clé) : Accessible uniquement par les classes du même package. Hérité uniquement si la sous-classe est dans le même package.
- *Gestion par Référence* : L'identificateur d'un objet stocke une adresse en mémoire. Si f2 = f1;, les deux variables pointent vers la même instance physique. Sa destruction est automatisée par le **Garbage Collector** (aucun destructeur requis).

Membres d'Instance vs Membres de Classe (static)

- **Membres d'Instance** : Propres à l'état de chaque objet individuel.
- **Membres de Classe (static)** : Partagés, uniques pour toute la classe, initialisés dès son chargement. Une méthode static (ex: Math.sqrt(), main()) **ne peut jamais** manipuler un attribut d'instance ou utiliser this.

Packages (Compilation et Exécution)

- Déclaration obligatoire en première ligne : package mon_package;.
- Le package **java.lang** est importé automatiquement.
- *Compilation d'un package* : Nécessite l'option -classpath de javac ou la configuration de la variable d'environnement **CLASSPATH**.
- *Exécution* : Exécuter depuis la racine avec le nom qualifié complet : java monPackage.MaClasse.

5. La Gestion des Exceptions

Une exception est un objet de la classe java.lang.Exception indiquant une anomalie d'exécution (IOException, FileNotFoundException).

Structure globale d'un bloc de capture :

- **try { ... }** : Regroupe les instructions à surveiller.
- **catch (TypeException e) { ... }** : Capture et traite spécifiquement l'erreur survenue.
- **finally { ... }** : Exécuté **inconditionnellement** (indispensable pour libérer des ressources comme fermer un fichier avec `f.close()`).

Déclaration de propagation (throws) et Déclenchement (throw)

- **throws** : Placé dans l'en-tête (la signature) d'une méthode pour avertir qu'elle ne traite pas l'erreur elle-même et qu'elle délègue sa gestion à la méthode appelante (Ex: `public void empiler(int x) throws ErreurPilePleine`).
- **throw new** : Provoque instantanément le déclenchement d'une exception dans le code (Ex: `throw new ErreurPilePleine();`).

FICHE DE RÉVISION COMPLÈTE - CHAPITRE 3 : Les Applets

1. Définition et Caractéristiques

- **Concept** : Une applet est un programme Java interactif stocké sur un serveur Web. Elle est téléchargée sur la machine cliente (le navigateur Web) et s'exécute directement à l'intérieur d'une page HTML.
- **Rôle** : Améliorer la présentation visuelle des pages Web ou remplir une fonction interactive spécifique.
- **Statut** : Les applets sont structurellement traitées comme des **objets** intégrés aux pages HTML qui les contiennent.
- **Architecture technique obligatoire** : La création et l'exécution d'une applet nécessitent obligatoirement la coexistence de **deux fichiers distincts** :
 1. Un fichier source de code Java (**.java**), compilé par la suite en bytecode (**.class**).
 2. Un fichier de structure Web (**.html**) qui appelle et intègre l'applet.

2. Restrictions de Sécurité (Le concept de "Bac à sable" / Sandbox)

Pour des raisons de sécurité évidentes (éviter qu'un code téléchargé sur Internet n'endommage l'ordinateur de l'utilisateur), une applet s'exécute dans un environnement restreint.

Il est STRICTEMENT INTERDIT à une applet de :

- Lire ou écrire sur le système de fichiers (le disque dur) de la machine cliente où elle s'exécute.
- Exécuter des programmes locaux ou des commandes système sur la machine hôte.
- Communiquer via le réseau avec d'autres machines, à l'**exception unique du serveur Web d'origine** à partir duquel elle a été téléchargée.

3. Architecture et Cycle de vie de la classe Applet

- **Héritage** : La classe Applet (appartenant au package `java.applet.Applet`) dérive directement de la classe `java.awt.Component`, qui est la classe de base d'infrastructure pour tous les objets graphiques en Java.
- **Méthodes d'affichage et d'interaction héritées/surchargées** :
 - `paint(Graphics g)` : Permet de dessiner des formes et du texte dans la fenêtre graphique de l'applet en exploitant l'objet de contexte Graphics (Ex : `g.drawString("Bonjour !", 50, 20);`).
 - `update(Graphics g)` : Gère le rafraîchissement de l'affichage dans la fenêtre.
 - `handleEvent(Event e)` : Gère les événements d'interaction utilisateur liés à la souris et au clavier.

Les 4 étapes majeures du Cycle de Vie (À savoir par cœur)

La classe Applet contient 4 méthodes spécifiques appelées automatiquement par le navigateur pour piloter son existence :

1. **init()** : Appelée une seule fois au moment exact du **chargement initial** de l'applet en mémoire. Idéale pour initialiser les variables et récupérer les paramètres.
2. **start()** : Appelée pour **démarrer ou redémarrer** l'exécution de l'applet (par exemple, lorsque l'utilisateur revient sur la page Web).
3. **stop()** : Appelée pour **suspendre** temporairement l'exécution de l'applet (par exemple, lorsque l'utilisateur change d'onglet ou quitte la page).
4. **destroy()** : Appelée à la fermeture définitive du navigateur pour **libérer les ressources** système monopolisées ou interrompre définitivement les activités de fond (comme un thread d'animation).

4. Intégration HTML et Passage de paramètres

La syntaxe de base de la balise <APPLET>

Pour insérer le programme dans une page Web, on utilise une balise spécifique. Lorsque le navigateur l'interprète, il télécharge le bytecode depuis le serveur.

```
<APPLET CODE="NomDuFichier.class" WIDTH=150 HEIGHT=25>
```

```
<!-- Le code s'exécute ici dans une zone de 150x25 pixels -->
```

```
</APPLET>
```

- **CODE** : Spécifie le nom du fichier binaire compilé (.class).
- **WIDTH / HEIGHT** : Définissent la taille initiale en pixels de la zone d'affichage allouée à l'applet.

Mécanisme d'injection de paramètres (PARAM et getParameter)

Il est possible de rendre une applet dynamique en lui transmettant des données depuis le fichier HTML. Ce mécanisme s'articule en deux étapes :

1. **Côté HTML** : Utilisation de la balise interne **<PARAM>** dotée des couples d'attributs name (identifiant) et value (valeur textuelle).

2. **Côté Java** : Appel de la méthode `getParameter("nom_du_param")` généralement confinée à l'intérieur du bloc `init()`. *Attention : cette méthode retourne toujours une chaîne de caractères (String).*

 **Exemple complet d'application (Modèle type examen) :**

Fichier : Bonjour.java

```
import java.awt.*;
import java.applet.*;

public class Bonjour extends Applet {
    String s1, s2;

    public void init() {
        // Récupération des chaînes injectées par le code HTML
        s1 = getParameter("param1");
        s2 = getParameter("param2");
    }

    public void paint(Graphics g) {
        // Affichage des textes combinés aux coordonnées X=50, Y=20
        g.drawString(s1 + " " + s2, 50, 20);
    }
}
```

Fichier : Bonjour.html

```
<HTML>
<BODY>
    <APPLET CODE="Bonjour.class" WIDTH=200 HEIGHT=100>
        <PARAM NAME="param1" VALUE="Bonjour">
        <PARAM NAME="param2" VALUE="Comment vas tu?">
    </APPLET>
</BODY>
</HTML>
```

QUIZ DU JOUR - SESSION 4 : Les Applets Java

Vérifions vos réflexes sur ce Chapitre 3 avant de clore la session !

Question 1 (Sécurité)

Une entreprise souhaite déployer une Applet Java sur son site intranet pour permettre aux employés de scanner un document et d'enregistrer automatiquement le fichier PDF directement sur le disque dur local (C:\Scans\) de leur poste de travail. Que se passe-t-il ?

- A) L'applet s'exécute normalement si elle utilise une méthode de `java.io.FileWriter`.
- B) L'exécution échoue immédiatement car, par sécurité, une applet a l'interdiction absolue de lire ou d'écrire sur le système de fichiers local du client.
- C) L'applet demande une confirmation de modification de base de données SQL avant d'écrire.
- D) Cela fonctionne uniquement si le navigateur utilise le package `javax.*`.

Question 2 (Cycle de vie)

Quelle méthode du cycle de vie d'une Applet est appelée automatiquement et de manière unique au tout début, spécifiquement lors de son chargement en mémoire par le navigateur ?

- A) `start()`
- B) `paint()`
- C) `init()`
- D) `destroy()`

Question 3 (Paramètres HTML)

Soit la ligne HTML suivante insérée dans une balise applet : `<PARAM NAME="age" VALUE="25">`. Comment devez-vous écrire le code dans la méthode `init()` de votre classe Java pour récupérer correctement cette valeur sous forme d'un entier exploitable ?

- A) `int x = getParameter("age");`
- B) `int x = Integer.parseInt(getParameter("age"));`
- C) `String x = getParameter(25);`
- D) `int x = (int) getParameter("age");`

Donnez-moi vos choix (ex: 1A, 2B, 3C).

FICHE DE RÉVISION UNIQUE - CHAPITRE 4 : Interfaces Graphiques (AWT & Swing)

1. Fondations des Interfaces Graphiques : AWT vs Swing

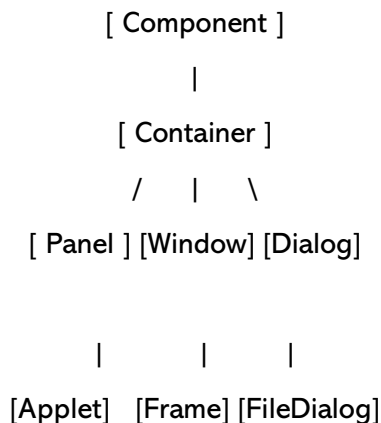
Le cours distingue deux boîtes à outils majeures pour la création d'interfaces en Java :

- **AWT (Abstract Window Toolkit) :**
 - Permet de créer des structures de fenêtres (Frame), des boutons (Button), des zones de texte (TextField), des listes, et d'exécuter des dessins.
 - **Caractéristique d'examen :** L'apparence visuelle des composants AWT dépend de la plate-forme d'exécution (ils utilisent les composants natifs du système d'exploitation hôte, appelés *peers*).

- **Swing :**
 - Propose des composants analogues à ceux d'AWT, préfixés par la lettre 'J' (JFrame, JButton, JTextField).
 - **Caractéristique d'examen :** Ils sont **indépendants de la plate-forme** (entièrement écrits en Java). Swing propose également des composants structurels avancés comme JTabbedPane (onglets) et JTree (arbres).

2. Hiérarchie des Composants et Conteneurs AWT

Sur table, vous devez connaître l'arborescence des classes d'infrastructure d'AWT :



- **java.awt.Component** : Classe abstraite racine de tous les objets graphiques (sauf les menus). Elle régit les comportements universels :
 - *Affichage* : paint(Graphics g).
 - *Géométrie* : Définir/obtenir la taille et la position (setSize(), setMinimumSize(), setBounds()).
 - *Aesthetic* : Configurer les couleurs et polices (setBackground(), setForeground(), setFont()).
 - *Événements* : Être une source d'événements.
- **java.awt.Container** : Sous-classe de Component. Espace graphique conçu pour recevoir et agglomérer d'autres composants graphiques via la méthode **add()**. Le placement physique de ces composants est délégué à un **gestionnaire de positionnement**.
- **java.awt.Window** : Sous-classe de Container. Représente un espace d'affichage autonome pouvant être visible à l'écran sans nécessiter d'être imbriqué dans un autre conteneur.
- **java.awt.Frame** : Sous-classe de Window. Fenêtre d'application standard dotée d'une bordure, d'un titre et éventuellement d'une barre de menus. Généralement, une application Java GUI instancie un cadre unique pour piloter la taille globale de l'interface.
- **java.awt.Panel** : Sous-classe de Container. Espace de regroupement intermédiaire. **Règle d'or sur table** : Un Panel n'est pas autonome, il doit obligatoirement être inséré dans un conteneur existant (autre conteneur ou navigateur Web pour une applet).

3. Le Modèle de Gestion des Événements (Modèle de Délégation)

La gestion des interactions repose sur l'interaction synchrone entre trois entités distinctes :

1. **L'Objet Source** : Le composant graphique sur lequel l'utilisateur agit (ex: un Button cliqué).
2. **L'Objet Événement** : Instance de classe (ex: `ActionEvent`, `MouseEvent`) générée par la source pour encapsuler les métadonnées de l'action (date, coordonnées, référence de la source).
3. **L'Écouteur (Listener)** : Objet implémentant une interface de supervision spécifique. Il doit s'enregistrer auprès de la source pour être notifié et traiter l'événement.

Les Interfaces d'Écouteurs Fondamentales

Chaque type d'interaction est surveillé par une interface dédiée rattachée au package `java.awt.event.*` :

- **ActionListener** : Capture les actions de déclenchement (clic de bouton, validation de zone de texte). Contient une unique méthode à implémenter : **`void actionPerformed(ActionEvent e)`**.
- **MouseListener** : Capture les événements d'état de la souris. Contient **5 méthodes obligatoires** à implémenter (même si elles restent vides) :
 - `void mousePressed(MouseEvent e)` / `void mouseReleased(MouseEvent e)`
 - `void mouseClicked(MouseEvent e)` (clic complet)
 - `void mouseEntered(MouseEvent e)` / `void mouseExited(MouseEvent e)` (survol de zone).
- **MouseMotionListener** : Détecte les mouvements de la souris (déplacement, glissement).
- **KeyListener** : Capture les frappes clavier (`keyPressed`, `keyReleased`).
- **FocusListener** : Détecte l'acquisition ou la perte de focus d'un champ par l'utilisateur.
- **WindowListener** : Traite les manipulations appliquées à la fenêtre (fermeture, réduction).

4. Les Gestionnaires de Positionnement (Layout Managers)

Plutôt que d'utiliser des coordonnées fixes, Java délègue le placement à des gestionnaires pour garantir l'indépendance visuelle vis-à-vis des plates-formes d'exécution.

A. FlowLayout

- **Principe** : Gestionnaire par défaut des conteneurs comme `Panel`. Il aligne les composants les uns derrière les autres sur une ligne. Si la ligne est pleine, il passe automatiquement à la ligne suivante.
- **Alignements** : Gérés par des constantes statiques : `FlowLayout.CENTER` (par défaut), `FlowLayout.LEFT`, `FlowLayout.RIGHT`.
- **Code type** : `setLayout(new FlowLayout(FlowLayout.LEFT));`

B. BorderLayout

- **Principe** : Découpe l'espace du conteneur en **5 zones géographiques** strictes.
- **Syntaxe d'ajout** : L'insertion impose de spécifier la zone cible : **`add(composant, BorderLayout.NOM_ZONE);`**

- **Les 5 zones** : NORTH, SOUTH, EAST, WEST, CENTER.
- *Note d'examen* : La zone CENTER s'étire automatiquement pour occuper tout l'espace central laissé libre par les autres zones.

C. GridLayout

- **Principe** : Divise l'espace en une **grille régulière de cases de dimensions strictement identiques**. Chaque composant ajouté prend la taille exacte de sa case de destination.
- **Fonctionnement** : Le constructeur exige deux arguments : **new GridLayout(nbLignes, nbColonnes)**. Les composants insérés remplissent la grille ligne par ligne, de gauche à droite.

5. Étude de Cas Pratiques : Code Modèle pour l'Examen

Cas 1 : Fenêtre Interactive de Multiplication (Modèle d'écriture sur table)

Ce code montre comment une classe fenêtre devient son propre écouteur via implements ActionListener.

```
import java.awt.*;
```

```
import java.awt.event.*;
```

// 1. La classe hérite de Frame et implémente l'interface de détection

```
public class Fenetre extends Frame implements ActionListener {
```

```
    public Panel p;
```

```
    public TextField texte;
```

```
    public Button bouton1, bouton2;
```

```
    public Fenetre() {
```

```
        super("Exemple de fenetre");
```

```
        p = new Panel(new BorderLayout()); // Utilisation d'un BorderLayout pour le panneau principal
```

```
        this.add(p);
```

// Insertion en zone Nord

```
p.add(new Label("Nbre a multiplier par 2"), BorderLayout.NORTH);
```

// Insertion en zone Centrale

```
p.add(texte = new TextField(20), BorderLayout.CENTER);
```

// Création d'un sous-panneau en FlowLayout pour regrouper les boutons au Sud

```

Panel p_sud = new Panel(new FlowLayout());
p.add(p_sud, BorderLayout.SOUTH);

p_sud.add(bouton1 = new Button("Doubler"));
p_sud.add(bouton2 = new Button("Fin"));

// 2. ENREGISTREMENT IMPÉRATIF : La fenêtre s'écoute elle-même
bouton1.addActionListener(this);
bouton2.addActionListener(this);

this.pack(); // Ajuste la taille de la fenêtre aux composants
this.setVisible(true); // Rend la fenêtre visible à l'écran
}

// 3. CODE DE TRAITEMENT : Implémentation obligatoire de la méthode de l'interface
public void actionPerformed(ActionEvent e) {
    if (e.getSource() == bouton1) {
        // Lecture du texte, conversion en réel, traitement et réécriture
        double x = Double.parseDouble(texte.getText());
        x = 2 * x;
        texte.setText("" + x);
    } else if (e.getSource() == bouton2) {
        this.dispose(); // Libère les ressources de la fenêtre
        System.exit(0); // Arrête proprement la Machine Virtuelle
    }
}

public static void main(String[] args) {
    new Fenetre(); // Instanciation et lancement
}
}

```

Cas 2 : Cadre Sensible aux Clics de Souris (Mécanisme MouseListener)

Ce code montre l'obligation d'implémenter toutes les signatures de l'interface, même si elles restent vides, et l'usage de `e.getX()` / `e.getY()` pour capturer les coordonnées du pointeur.

```

import java.awt.*;
import java.awt.event.*;

public class CadreSensible extends Frame implements MouseListener {

    public CadreSensible() {
        addMouseListener(this); // Enregistrement de l'écouteur sur le cadre
        this.setBounds(100, 100, 300, 200); // Position X=100, Y=100, Largeur=300,
        Hauteur=200
        this.setVisible(true);
    }

    // Capture du clic complet (Enfoncé puis Relâché au même endroit)
    public void mouseClicked(MouseEvent e) {
        System.out.println("Clic en (" + e.getX() + "," + e.getY() + ")");
    }

    // Obligations syntaxiques : Les autres méthodes doivent être écrites à vide
    public void mousePressed(MouseEvent e) {}
    public void mouseReleased(MouseEvent e) {}
    public void mouseEntered(MouseEvent e) {}
    public void mouseExited(MouseEvent e) {}

    public static void main(String args[]) {
        new CadreSensible();
    }
}

```

QUIZ ÉCLAIR - CHAPITRE 4 : Interfaces Graphiques

Testons vos réflexes sur ce dernier bloc d'aujourd'hui !

Question 1 (Hiérarchie)

À l'examen, on vous donne ce bloc de code : `Panel monPanneau = new Panel(); monPanneau.setVisible(true);`. Bien qu'aucune erreur de compilation ne surgisse, le panneau reste invisible à l'écran. Pourquoi ?

- A) Il faut obligatoirement appeler `monPanneau.pack();` d'abord.

- B) Un Panel est un conteneur qui n'est pas autonome ; il doit obligatoirement être inséré dans un autre conteneur existant (comme une Frame) pour s'afficher.
- C) La méthode setVisible n'existe pas pour la classe Panel.
- D) Les panneaux ne fonctionnent que sous Swing (JPanel).

Question 2 (Gestionnaires de positionnement)

Vous instanciez un conteneur configuré de la sorte : `setLayout(new GridLayout(3, 2));`. Vous y ajoutez successivement 5 boutons via la méthode `add()`. Quelle sera la structure physique finale de l'interface ?

- A) Les boutons seront empilés sur une seule colonne de 5 cases.
- B) Les boutons seront répartis sur 3 lignes : les deux premières lignes contiendront 2 boutons chacune, et la troisième ligne contiendra le 5ème bouton. Les cases feront toutes la même taille.
- C) Les 3 premiers boutons occuperont la première ligne, et les 2 suivants la deuxième ligne.
- D) Le code génère une exception à l'exécution car la grille doit être totalement remplie (6 boutons requis).

Question 3 (Syntaxe des Événements)

Vous écrivez une classe qui implémente l'interface `MouseListener` pour détecter uniquement le survol d'une zone via la méthode `mouseEntered()`. Vous laissez les 4 autres méthodes de l'interface en dehors de votre fichier de code car vous n'en avez pas besoin. Que se passe-t-il à la compilation ?

- A) Le code compile normalement. Java ignore les méthodes vides.
- B) Le compilateur renvoie une erreur obligatoire indiquant que votre classe doit être déclarée abstraite ou implémenter *toutes* les méthodes définies par l'interface `MouseListener`.
- C) Une exception de type `IOException` est levée.
- D) L'application compile mais plante dès que l'utilisateur déplace la souris.

Donnez-moi vos choix (ex: 1A, 2B, 3C).

Avant de synthétiser le Chapitre 5, voici vos résultats pour le mini-quiz du Chapitre 4 :

- **Question 1 : B** (Un Panel doit obligatoirement être inséré dans un conteneur parent comme une Frame pour apparaître).
- **Question 2 : B** (Le placement se fait ligne par ligne, donc 2 sur la ligne 1, 2 sur la ligne 2, et le dernier sur la ligne 3).
- **Question 3 : B** (Le compilateur refuse une implémentation partielle d'une interface, sauf si la classe est déclarée abstraite).

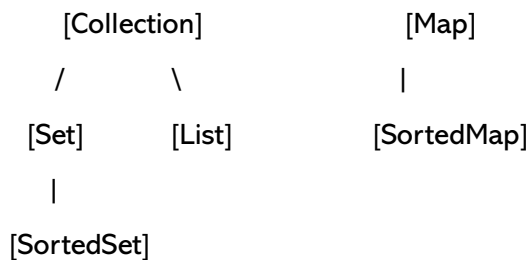
FICHE DE RÉVISION UNIQUE - CHAPITRE 5 : Collections Java & Connectivité Base de Données (JDBC)

PARTIE I : Les Collections Java

L'infrastructure des collections de Java (java.util.*) fournit un ensemble d'interfaces et de classes conçues pour stocker et manipuler des groupes d'objets.

1. La Hiérarchie des Interfaces

Les structures se divisent en deux catégories distinctes à la racine :



- **Collection** : Groupe d'objets générique où les doublons peuvent être autorisés. Elle spécifie les méthodes universelles : add(), remove(), addAll(), removeAll(), contains(), containsAll(), size().
- **Set** : Collection d'objets **non ordonnés et sans aucun doublon**.
 - **SortedSet** : Un Set dont les éléments sont triés automatiquement.
- **List** : Collection **ordonnée qui autorise les doublons**. Chaque élément possède une position (index numérique) dans la séquence. Elle ajoute des méthodes d'accès direct positionnel.
- **Map** : Tableaux associatifs (ou dictionnaires) structurés sous forme de paires (**clé, valeur**), où chaque clé est unique (sans doublon). Elle permet de retrouver une valeur grâce à son index de clé.
 - **SortedMap** : Un Map trié automatiquement selon l'ordre de ses clés.

2. Matrice des Implémentations Officielles (À savoir par cœur)

Interface	Table de Hachage	Tableau (Vecteur)	Arbre	Liste Chaînée
Set	HashSet	—	TreeSet	—
List	—	ArrayList	—	LinkedList
Map	HashMap	—	TreeMap	—

- *Note d'examen* : Les structures basées sur un **Arbre** (TreeSet, TreeMap) implémentent les versions triées (SortedSet, SortedMap).

3. Le Parcours par Itérateur (Iterator)

Toutes les classes dérivées de l'interface Collection peuvent générer un objet **Iterator** pour parcourir leurs éléments de manière sécurisée sans boucle indexée classique.

- `hasNext()` : Retourne un booléen indiquant s'il reste des éléments à parcourir.
- `next()` : Récupère l'élément suivant. *Attention : cette méthode retourne un type `Object` brut, il faut appliquer un casting manuel pour retrouver le type d'origine.*

```
Set ufrs = new TreeSet(); // Arbre trié
```

```
ufrs.add("sat"); ufrs.add("seg");
```

```
Iterator iter = ufrs.iterator();
```

```
while (iter.hasNext()) {
```

```
    String ufr = (String) iter.next(); // Casting obligatoire de Object vers String
```

```
    System.out.println(ufr);
```

```
}
```

4. Spécificités et Itérations sur les objets Map

Un Map manipule des paires. L'interface interne **Map.Entry** permet de contrôler chaque paire au moyen de :

- `getKey()` / `getValue()` : Retournent respectivement la clé et la valeur associée.
- `setValue()` : Permet de modifier la valeur d'une paire existante.

 **Les 3 méthodes d'itérations d'un Map (Sujet d'examen très fréquent) :**

```
Map m = new HashMap();
```

```
// A. Itération sur les Clés uniquement (.keySet())
```

```
for (Iterator i = m.keySet().iterator(); i.hasNext();)
```

```
    System.out.println(i.next());
```

```
// B. Itération sur les Valeurs uniquement (.values())
```

```
for (Iterator i = m.values().iterator(); i.hasNext();)
```

```
    System.out.println(i.next());
```

```
// C. Itération sur les Paires clé/valeur (.entrySet())
```

```
for (Iterator i = m.entrySet().iterator(); i.hasNext();) {
```

```
    Map.Entry e = (Map.Entry) i.next(); // Casting obligatoire en Map.Entry
```

```
    System.out.println(e.getKey() + "/" + e.getValue());
```

```
}
```

- *Attention à la casse du cours : Le cours écrit syntaxiquement `EntrySet()` par erreur, la méthode standard de l'API s'écrit `entrySet()`.*

PARTIE II : JDBC (Java Database Connectivity)

L'API JDBC (java.sql.*) permet à une application Java de se connecter en mode **client/serveur** à n'importe quel Système de Gestion de Bases de Données (SGBD) du marché (MySQL, Oracle, PostgreSQL, MS Access, SQL Server).

1. L'Architecture JDBC

L'architecture repose sur une séparation des rôles en deux couches logicielles :

1. **JDBC Driver Manager** : Gestionnaire de pilotes central de Java. Il sert d'interface de liaison globale et permet à l'application de charger dynamiquement le ou les pilotes dont elle a besoin.
2. **Driver JDBC (Pilote)** : Composant logiciel spécifique qui gère les détails techniques de communication réseau propres à un SGBD donné. Il existe **un driver dédié par SGBD**.
 - **JDBC-ODBC** : Driver générique d'infrastructure permettant d'accéder à toutes les sources de données configurées via l'interface Microsoft ODBC (Open DataBase Connectivity) sous Windows.

2. Cycle de fonctionnement Standard de JDBC

Pour manipuler des données, une application suit obligatoirement 4 étapes chronologiques :

1. Créer la connexion → 2. Envoyer la requête SQL → 3. Exploiter les résultats → 4. Fermer la connexion

Code Modèle : Connexion et Interrogation (À savoir réécrire à la main)

```
import java.sql.*;
```

```
public class TestJDBC {  
    public static void main(String[] args) {  
        Connection co = null;  
        String url = "jdbc:mysql://localhost/MABD"; // URL de connexion réseau  
        String req = "Select nom, sexe, age from Personne";  
  
        try {  
            // ÉTAPE 1 : Chargement dynamique de la classe du Driver MySQL par réflexion  
            Class.forName("org.gjt.mm.mysql.Driver");  
  
            // ÉTAPE 2 : Établissement de la connexion  
            co = DriverManager.getConnection(url);  
  
            // ÉTAPE 3 : Création de la passerelle de requête et exécution
```

```

Statement st = co.createStatement();

ResultSet rs = st.executeQuery(req); // executeQuery est réservé aux SELECT

// ÉTAPE 4 : Parcours séquentiel des lignes du résultat
while (rs.next()) {
    // Accès aux colonnes par leur index numérique (base 1) ou par leur nom de champ
    System.out.print("nom :" + rs.getString(1));    // Colonne 1
    System.out.print("sexe:" + rs.getString("sexe")); // Nom de colonne
    System.out.println("age :" + rs.getDouble(3));   // Colonne 3
}

// ÉTAPE 5 : Libération impérative des ressources dans l'ordre inverse
rs.close();
st.close();
co.close();

} catch (ClassNotFoundException e) {
    System.out.println("Driver introuvable");
} catch (SQLException se) {
    System.out.println("Erreur SQL ou connexion impossible");
}
}
}

```

3. Les Différents Types d'Objets Requêtes (Statements)

Le cours répertorie trois structures d'objets pour formuler des ordres SQL :

1. **Statement** : Utilisé pour envoyer des requêtes SQL standards simples et textuelles, sans paramètres dynamiques.
2. **PreparedStatement** : Correspond à une requête SQL **précompilée** et gérée au niveau du client JDBC. Elle peut être paramétrée (avec des points d'interrogation ?) et exécutée à de multiples reprises de manière très performante.
3. **CallableStatement** : Correspond à une requête SQL stockée, écrite directement dans le langage procédural natif de la base de données (Stored Procedures) et **gérée directement par le SGBD**. Elle peut également être paramétrée.

4. Gestion des Métadonnées (Metadata)

Les métadonnées permettent à l'application Java d'inspecter dynamiquement la structure de la base de données et des tables sans en connaître le contenu à l'avance.

- **ResultSet** : Objet retourné par `executeQuery()`. Il contient les lignes de données du résultat et permet de se déplacer de ligne en ligne via la méthode `rs.next()`.
 - **ResultSetMetaData** : Contient les informations structurelles sur le résultat d'une table (les noms des colonnes, le nombre de colonnes, leurs types de données).
 - *Obtention* : `ResultSetMetaData rsmd = rs.getMetaData();`
 - *Méthode clé* : `int nbre_Colonne = rsmd.getColumnCount();`
 - **DatabaseMetaData** : Contient les informations globales sur la base de données elle-même (le nom du SGBD, la liste des tables présentes, les index, etc.).
 - *Obtention* : `DatabaseMetaData dbmd = co.getMetaData();`
-

QUIZ ÉCLAIR - CHAPITRE 5 : Collections & JDBC

Question 1 (Collections)

Vous devez stocker la liste des adresses IP des serveurs de votre infrastructure Master. Vous voulez vous assurer qu'**aucun doublon** ne soit enregistré, et vous avez besoin que la liste soit **automatiquement triée** par ordre alphanumérique pour faciliter les recherches. Quelle classe d'implémentation devez-vous instancier ?

- A) ArrayList
- B) HashMap
- C) TreeSet
- D) HashSet

Question 2 (JDBC - Architecture)

Quel composant de l'architecture JDBC sert d'interface de liaison globale pour charger dynamiquement en mémoire le pilote réseau d'un SGBD (comme le driver MySQL) ?

- A) ResultSetMetaData
- B) PreparedStatement
- C) JDBC Driver Manager
- D) CallableStatement

Question 3 (JDBC - Code)

Soit un objet `ResultSet rs` valide obtenu après l'exécution d'un script `SELECT`. Quelle méthode devez-vous impérativement appeler dans votre boucle `while` pour faire avancer le curseur de lecture sur la ligne suivante et vérifier que des données sont bien présentes ?

- A) `rs.close()`
 - B) `rs.next()`
 - C) `rs.getMetaData()`
 - D) `rs.getString(1)`
-

Donnez-moi vos choix (ex: 1A, 2B, 3C).

